

ЛАБОРАТОРНАЯ РАБОТА № 4

Процессы

Ход выполнения лабораторной работы

- Задание для выполнения лабораторной работы Составить следующую программу, которая: принимая дескриптор, имя или полное имя модуля, возвращает другие два элемента в своих выходных параметрах (выполнить задание для своей программы и для любой известной библиотеки),будет выполнять последовательно по шагам следующее:
1. используя функцию GetCurrentProcessId определит идентификатор текущего процесса.
 2. используя функцию GetCurrentProcess определит псевдодескриптор текущего процесса.
 3. используя функцию DuplicateHandle и значение псевдодескриптора определит дескриптора текущего процесса.
 4. используя функцию OpenProcess определит копию дескриптора текущего процесса
 5. закроет дескриптор, полученный функцией DuplicateHandle
 6. закроет дескриптор, полученный функцией OpenProcess
 - выдает список перечисления всех процессов, потоков, модулей и их свойств.

```
#include <windows.h>
#include <tlhelp32.h>
#include <psapi.h>
#include <iostream>

DWORD GetCurrentProcessId() {
    return GetCurrentProcessId();
}

HANDLE GetCurrentProcessHandle() {
    return GetCurrentProcess();
}

HANDLE DuplicateProcessHandle(HANDLE hProcess) {
    HANDLE hDuplicate = NULL;
    if (DuplicateHandle(GetCurrentProcess(), hProcess, GetCurrentProcess(), 0, FALSE, DUPLICATE_SAME_ACCESS)) {
        return hDuplicate;
    }
    else {
        std::cerr << "failed to duplicate handle. Error: " << GetLastError() << std::endl;
        return NULL;
    }
}

HANDLE openProcess(DWORD processId) {
    HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ, FALSE, processId);
    if (hProcess == NULL) {
        std::cerr << "failed to open process. Error: " << GetLastError() << std::endl;
    }
    return hProcess;
}

void CloseHandle(HANDLE hProcess) {
    if (!CloseHandle(hProcess)) {
        std::cerr << "failed to close handle. Error: " << GetLastError() << std::endl;
    }
}

HANDLE CreateSnapshot(DWORD flags) {
    HANDLE hSnapshot = CreateToolhelp32Snapshot(flags, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE) {
        std::cerr << "failed to create snapshot. Error: " << GetLastError() << std::endl;
    }
    return hSnapshot;
}

void ListProcesses() {
    HANDLE hSnapshot = CreateSnapshot(TH32CS_SNAPPROCESS);
    if (hSnapshot == INVALID_HANDLE_VALUE) {

```

```
        return;
    }
    PROCESSENTRY32 pe32;
    pe32.dwSize = sizeof(PROCESSENTRY32);
    if (!Process32First(hSnapshot, &pe32)) {
        std::cerr << "failed to get first process. Error: " << GetLastError() << std::endl;
        CloseHandle(hSnapshot);
        return;
    }
    do {
        std::cout << "Process Name: " << pe32.szExeFile << " | Process ID: " << pe32.th32ProcessID << std::endl;
    } while (Process32Next(hSnapshot, &pe32));
    CloseHandle(hSnapshot);

    void ListModules(DWORD processId) {
        HANDLE hSnapshot = CreateSnapshot(TH32CS_SNAPMODULE | TH32CS_SNAPMODULE32);
        if (hSnapshot == INVALID_HANDLE_VALUE) {
            return;
        }
        MODULEENTRY32 me32;
        me32.dwSize = sizeof(MODULEENTRY32);
        if (!Module32First(hSnapshot, &me32)) {
            std::cerr << "failed to get first module. Error: " << GetLastError() << std::endl;
            CloseHandle(hSnapshot);
            return;
        }
        do {
            std::cout << "Module Name: " << me32.szModule << " | Module Path: " << me32.szExePath << std::endl;
        } while (Module32Next(hSnapshot, &me32));
        CloseHandle(hSnapshot);

    void GetProcessMemoryInfo(DWORD processId) {
        HANDLE hProcess = OpenProcess(processId);
        if (hProcess == NULL) {
            return;
        }
        PROCESS_MEMORY_COUNTERS pmc;
        if (GetProcessMemoryInfo(hProcess, &pmc, sizeof(pmc))) {
            std::cout << "Process ID: " << processId << std::endl;
            std::cout << "Working Set Size: " << pmc.WorkingSetSize << std::endl;
            std::cout << "Pagefile Usage: " << pmc.PagefileUsage << std::endl;
        }
        else {
            std::cerr << "failed to get memory info for process with ID: " << processId << std::endl;
        }
        CloseHandle(hProcess);
    }
}
```

```

int main() {
    DWORD processId = GetCurrentProcessId();
    std::cout << "Current Process ID: " << processId << std::endl;

    HANDLE hProcess = GetCurrentProcessHandle();
    std::cout << "Current Process Handle: " << hProcess << std::endl;

    HANDLE hDuplicate = duplicateProcessHandle(hProcess);
    if (hDuplicate != NULL) {
        std::cout << "Duplicate Process Handle: " << hDuplicate << std::endl;

        HANDLE hOpenProcess = openProcess(processId);
        if (hOpenProcess != NULL) {
            std::cout << "Open Process Handle: " << hOpenProcess << std::endl;

            closeHandle(hDuplicate);
            std::cout << "Closed Duplicate Handle" << std::endl;

            closeHandle(hOpenProcess);
            std::cout << "Closed Open Process Handle" << std::endl;
        }
    }

    listProcesses();

    listModules(processId);

    getProcessMemoryInfo(processId);

    return 0;
}

```

```

Korczak on Visual Studio
Process Name: svchost.exe | Process ID: 111222
Process Name: SearchProtocolHost.exe | Process ID: 111144
Process Name: SearchFilterHost.exe | Process ID: 186764
Process Name: msedgebrowser.exe | Process ID: 111844
Process Name: msedgebrowser.exe | Process ID: 91164
Process Name: msedgebrowser.exe | Process ID: 111488
Process Name: msedgebrowser.exe | Process ID: 111572
Process Name: msedgebrowser.exe | Process ID: 187392
Process Name: msedgebrowser.exe | Process ID: 94544
Process Name: msedgebrowser.exe | Process ID: 186852
Process Name: ConsoleApplication1.exe | Process ID: 7928
Process Name: msimon.exe | Process ID: 79356
Module Name: ConsoleApplication1.exe | Module Path: C:\Users\Korziwe\source\repos\ConsoleApplication1.exe
Module Name: nt.dll | Module Path: C:\Windows\SYSTEM32\ntdll.dll
Module Name: KERNEL32.DLL | Module Path: C:\Windows\System32\KERNEL32.DLL
Module Name: KERNELBASE.dll | Module Path: C:\Windows\System32\KERNELBASE.dll
Module Name: MSVCPU1400.dll | Module Path: C:\Windows\SYSTEM32\MSVCPU1400.dll
Module Name: VCRUNTIME140_ID.dll | Module Path: C:\Windows\SYSTEM32\VCRUNTIME140_ID.dll
Module Name: VCRUNTIME1400.dll | Module Path: C:\Windows\SYSTEM32\VCRUNTIME1400.dll
Module Name: ucrtbased.dll | Module Path: C:\Windows\SYSTEM32\ucrtbased.dll
Process ID: 7928
Working Set Size: 3837952
Pagefile Usage: 692224

```